grist-kit

Release 0.1.2

May 19, 2026

Contents

1 CLI **2**
- 1.1 Quick Start . . . 2
- 1.2 Configuration . . . 3
  - Environment variables . . . 3
  - Flags . . . 3
- 1.3 Listing Tables . . . 3
  - Text output . . . 3
  - JSON output . . . 4
- 1.4 Fetching Records . . . 4
  - records . . . 4
  - sql . . . 5

2 Type-Safe Client Library **5**
- 2.1 Installation . . . 6
- 2.2 Generating a Type Definition File . . . 6
- 2.3 Creating a Client . . . 6
  - Prerequisites . . . 6
  - Instantiating . . . 6
  - API reference . . . 7
- 2.4 Working with Records . . . 7
  - list . . . 7
  - insert . . . 8
  - update . . . 8
  - upsert . . . 8
  - delete . . . 8
  - API reference . . . 8
- 2.5 Working with Attachments . . . 9
  - upload . . . 9
  - get . . . 9
  - download . . . 9

downloadStream 9
API reference 9

3 Full Example **9**
3.1 Setup 10
3.2 The script 10
3.3 Run it 11

A CLI and type-safe client library for Grist[1], written in TypeScript. It provides:

- An agent-friendly *CLI* (page 2) for interacting with Grist documents.
- A *command-line tool to generate type definitions* (page 6) from a Grist table schema, along with a *type-safe client* (page 6) for programmatically querying and manipulating Grist documents.

---

1 CLI

grist-kit provides a command-line interface for working with Grist documents — *listing tables* (page 3), *fetching records, and running SQL queries* (page 4) — without writing code or making manual API calls. This makes it useful for both humans and AI agents that need to inspect or manipulate Grist data directly from the shell.

1.1 Quick Start

You can query any public Grist document with just `--doc-url` — no configuration needed. The examples below use the public Inventory Manager[2] template.

List the tables and their columns:

```
npx -y grist-kit --doc-url
↪https://templates.getgrist.com/api/docs/sXsBGDTKau1F3fvxkCyoaJ tables
```

Fetch records as CSV:

```
npx -y grist-kit --doc-url
↪https://templates.getgrist.com/api/docs/sXsBGDTKau1F3fvxkCyoaJ \
  records All_Products --limit 3 --format csv
```

```
id,SKU,Product,In_Stock,Stock_Alert,...
1,VEG-BLCK-28,Men's Stretch Five-Pocket Pants,8,In Stock,...
2,VEG-BLCK-30,Men's Stretch Five-Pocket Pants,8,In Stock,...
3,VEG-BLCK-32,Men's Stretch Five-Pocket Pants,9,In Stock,...
```

Filter by a column value:

[1] https://www.getgrist.com

[2] https://templates.getgrist.com/sXsBGDTKau1F/Inventory-Manager

```
npx -y grist-kit --doc-url
↪https://templates.getgrist.com/api/docs/sXsBGDTKau1F3fvxkCyoaJ \
  records All_Products --filter Stock_Alert="Low Stock" --format csv
```

```
id,SKU,Product,In_Stock,Stock_Alert,...
5,VEG-BLCK-36,Men's Stretch Five-Pocket Pants,2,Low Stock,...
6,VEG-BLCK-38,Men's Stretch Five-Pocket Pants,2,Low Stock,...
...
```

1.2 Configuration

All CLI commands require a Grist document URL. Credentials are read from environment variables or flags.

Environment variables

`GRIST_DOC_URL`
: Base doc URL — e.g. `https://grist.example.com/api/docs/xxxx`. Required for all commands.

`GRIST_API_KEY`
: Grist API key. Required unless the document is public.

The CLI reads from environment variables. It also supports loading a `.env` file, though real environment variables take precedence.

Flags

These flags are accepted by every command and override the corresponding environment variable.

`--doc-url <url>`
: Overrides `GRIST_DOC_URL`.

`--api-key <key>`
: Overrides `GRIST_API_KEY`.

`--env-file <path>`
: Load a specific `.env` file instead of the default.

1.3 Listing Tables

The `tables` command lists all tables in a Grist document, along with their columns and types.

```
npx -y grist-kit tables [options]
```

`--format <text|json>`
: Output format. Defaults to `text`.

Text output

```
npx -y grist-kit tables
```

```
All_Products
  SKU Text
  Product Text
  Size Choice
  In_Stock Numeric
  Stock_Alert Choice
  Brand Ref:Add_Products
  Color Ref:Color
  ...
Incoming_Orders
  Order_Date Date
  Status Choice
  Total Numeric
  ...
```

JSON output

Use `--format json` to get the full column metadata from the Grist API, including labels, formulas, and widget options. Useful for inspecting the document schema.

```
npx -y grist-kit tables --format json
```

```
[
  {
    "id": "All_Products",
    "fields": { "tableRef": 1, "primaryViewId": 1, ... },
    "columns": [
      {
        "id": "SKU",
        "fields": {
          "label": "SKU",
          "type": "Text",
          "isFormula": false,
          "formula": "",
          ...
        }
      },
      ...
    ]
  },
  ...
]
```

1.4 Fetching Records

records

Fetches rows from a table and prints them as JSON.

```
npx -y grist-kit records <table> [options]
```

`--filter <key=value>`
: Filter rows by column value. Repeatable for multiple filters.

`--limit <n>`
: Maximum number of rows to return.

`--format <json|csv>`
: Output format. Defaults to `json`.

Examples:

```
npx -y grist-kit records All_Products --limit 2
```

```
[
  {
    "id": 1,
    "SKU": "VEG-BLCK-28",
    "Product": "Men's Stretch Five-Pocket Pants",
    "In_Stock": 8,
    "Stock_Alert": "In Stock"
  },
  ...
]
```

```
npx -y grist-kit records All_Products --filter Stock_Alert=Low Stock --format csv
```

sql

Runs a SQL query against the document and prints results.

```
npx -y grist-kit sql "<query>" [options]
```

`--format <json|csv>`
: Output format. Defaults to `json`.

Examples:

```
npx -y grist-kit sql "SELECT SKU, In_Stock FROM All_Products ORDER BY In_Stock LIMIT
↪3" --format csv
```

```
SKU,In_Stock
VEG-BLCK-34,0
SEW-LTBL-36,0
SEW-LTBL-28,0
```

2 Type-Safe Client Library

The grist-kit client library lets you query and manipulate Grist documents from TypeScript with full type safety. Column names, filter values, and write payloads are all checked against a generated schema at compile time.

2.1 Installation

Install grist-kit as a dependency in your project:

```
npm install grist-kit
```

2.2 Generating a Type Definition File

Before using the type-safe client, you need to generate a TypeScript type definition file from your live Grist document. This file describes your tables and columns, and is passed as a type parameter to `gristDoc` to enable typesafe column access, filters, and inserts.

The `generate` command reads the document schema via the Grist API and writes the type definition file. See *Configuration* (page 3) for how to configure the document URL and API key.

```
npx grist-kit generate [options]
```

`--out <path>`
: Output file path. Defaults to `./grist-schema.ts`.

`--type-name <name>`
: Name of the exported TypeScript type. Defaults to `GristSchema`.

Example:

```
npx grist-kit generate --out grist-schema.ts
```

Re-run this command whenever the document's columns change.

> **Tip**
>
> Save it as a `package.json` script so it's easy to remember:
>
> ```
> pnpm pkg set scripts.update-schema="grist-kit generate --out grist-schema.ts"
> ```

2.3 Creating a Client

Prerequisites

Before using the client, generate a schema file from your live doc:

```
npx grist-kit generate --out grist-schema.ts
```

See *Generating a Type Definition File* (page 6) for details.

Instantiating

```
import { gristDoc } from "grist-kit";
import type { GristSchema } from "./grist-schema.ts";
```

(continues on next page)

(continued from previous page)

```
const doc = gristDoc<GristSchema>({
  baseDocUrl: process.env.GRIST_DOC_URL!,
  apiKey: process.env.GRIST_API_KEY,
});
```

Pass the generated schema as the type parameter to enable type inference across all table and column operations.

baseDocUrl
: Base URL of the Grist document. Required.

apiKey
: Grist API key. Required unless the document is public.

fetchOptions
: Additional options forwarded to the underlying `ofetch` client — useful for custom headers or timeouts.

API reference

- gristDoc[3] — factory function
- GristDoc[4] — document client

2.4 Working with Records

Access a table via `doc.table("TableName")`. All methods are typed against the generated schema.

list

Fetches rows matching the given options.

```
const products = await doc.table("All_Products").list({
  filter: { Stock_Alert: ["Low Stock", "OUT OF STOCK"] },
  sort: "-In_Stock",
  limit: 100,
});
```

filter
: Equality-style filters keyed by column name. Each value is an array of allowed values.

sort
: Column name to sort by. Prefix with `-` for descending order. Accepts a single value or an array.

limit
: Maximum number of rows to return.

hidden
: Include hidden columns in the response.

[3] https://apiref-site.vercel.app/package/grist-kit/gristDoc

[4] https://apiref-site.vercel.app/package/grist-kit/GristDoc

insert

Inserts one or more rows and returns their row IDs.

```
const [orderId] = await doc
  .table("Incoming_Orders")
  .insert([{ Order_Date: Math.floor(Date.now() / 1000), Status: "Order Placed" }]);
```

Formula columns are excluded from the insert payload type automatically.

update

Updates existing rows by ID.

```
await doc.table("Incoming_Orders").update([{ id: orderId, Status: "Received" }]);
```

upsert

Creates or updates rows using Grist's upsert endpoint. Each record specifies `require` (match criteria) and `fields` (values to write).

```
await doc
  .table("Customers")
  .upsert([{ require: { Email: "alice@example.com" }, fields: { Name: "Alice", Tier:
↪"pro" } }]);
```

onMany
: Behavior when multiple rows match: `"first"`, `"none"`, or `"all"`.

noCreate
: Do not create a new row when no match is found.

noUpdate
: Do not update an existing row when a match is found.

delete

Deletes rows by row ID.

```
await doc.table("Incoming_Orders").delete([orderId]);
```

API reference

- GristTable[5] — table client
- GristTable.list[6]

[5] https://apiref-site.vercel.app/package/grist-kit/GristTable
[6] https://apiref-site.vercel.app/package/grist-kit/GristTable/list

2.5 Working with Attachments

Attachment operations are available via `doc.attachments`.

upload

Uploads one or more files and returns their attachment IDs. These IDs can be stored in `Attachments`-typed columns.

```
const [id] = await doc.attachments.upload([
  { filename: "report.pdf", data: pdfBuffer, type: "application/pdf" },
]);
```

Accepts `File`, `Blob`, or an object with `filename`, `data` (a `Blob`, `Uint8Array`, or `ArrayBuffer`), and an optional `type`.

get

Retrieves metadata for an attachment by ID.

```
const meta = await doc.attachments.get(id);
console.log(meta.fileName, meta.fileSize, meta.timeUploaded);
```

download

Downloads an attachment as a `Blob`.

```
const blob = await doc.attachments.download(id);
```

downloadStream

Downloads an attachment as a readable byte stream.

```
const stream = await doc.attachments.downloadStream(id);
```

API reference

- GristAttachments[7] — attachment client

3 Full Example

This example shows a complete script that connects to a Grist doc, reads low-stock products, and places a refill order. It uses the Inventory Manager[8] template and covers the full workflow: setup, type generation, and a typesafe read/write script.

[7] https://apiref-site.vercel.app/package/grist-kit/GristAttachments

[8] https://www.getgrist.com/templates/inventory-manager/

Prerequisites: Node.js 24+[9] and pnpm[10].

3.1 Setup

Create a project and install grist-kit:

```
mkdir grist-inventory && cd grist-inventory
pnpm init --init-type module
pnpm add grist-kit
pnpm add -D typescript @types/node @tsconfig/node24
```

Create `tsconfig.json`:

```
{
  "extends": "@tsconfig/node24/tsconfig.json"
}
```

Open the Inventory Manager template doc[11]. In the left sidebar, click **Settings** → **API** → expand **Document ID** → copy the **Base doc URL**.

Create `.env`:

```
GRIST_DOC_URL=https://templates.getgrist.com/api/docs/sXsBGDTKau1F3fvxkCyoaJ
```

Generate types from the live doc:

```
pnpm exec grist-kit generate --out grist-schema.ts
```

3.2 The script

Create `refill.ts`:

```
import { parseArgs } from "node:util";
import { gristDoc } from "grist-kit";
import type { GristSchema } from "./grist-schema.ts";

const { values } = parseArgs({
  options: { "dry-run": { type: "boolean", default: false } },
});

const doc = gristDoc<GristSchema>({
  baseDocUrl: process.env.GRIST_DOC_URL!,
  apiKey: process.env.GRIST_API_KEY,
});

const needsRefill = await doc.table("All_Products").list({
  filter: { Stock_Alert: ["Low Stock", "OUT OF STOCK"] },
});
```

(continues on next page)

[9] https://nodejs.org/docs/latest/api/typescript.html

[10] https://pnpm.io/

[11] https://templates.getgrist.com/sXsBGDTKau1F/Inventory-Manager

(continued from previous page)

```
console.log(`${needsRefill.length} products need a refill:\n`);
for (const p of needsRefill) {
  console.log(`  [${p.Stock_Alert}] ${p.SKU} - ${p.Product} (in stock: ${p.In_Stock}
↪)`);
}

if (needsRefill.length === 0 || values["dry-run"]) {
  if (values["dry-run"]) console.log("\n(dry run - no order created)");
  process.exit(0);
}

const [orderId] = await doc
  .table("Incoming_Orders")
  .insert([{ Order_Date: Math.floor(Date.now() / 1000), Status: "Order Placed" }]);

await doc.table("Incoming_Order_Line_Items").insert(
  needsRefill.map((p) => ({
    Order_Number: orderId,
    SKU: p.id,
    Qty: 10,
  })),
);

console.log(`\nCreated order ${orderId} with ${needsRefill.length} line items.`);
```

A few things worth noticing:

- The `filter` argument is fully typed against the schema. Try misspelling `"OUT OF STOCK"` — TypeScript will reject it, because `Stock_Alert`'s allowed values are baked into the generated schema.
- `In_Stock` and `Stock_Alert` are formula columns. The schema marks them as such, so they don't appear in the `insert()` payload type — you can't accidentally try to write them.
- `Order_Date` is a Grist `Date`, encoded as epoch seconds.
- Every Grist row has a numeric `id`. `Order_Number` is a `Ref` to `Incoming_Orders`, so it expects that row id.

3.3 Run it

Test against the public template (read-only, no API key needed):

```
node --env-file=.env refill.ts --dry-run
```

To actually place the order, make a copy of the template doc under your own account, generate an API key in **Profile Settings** → **API**, and update `.env`:

```
GRIST_DOC_URL=https://<your-host>/api/docs/<your-doc-id>
GRIST_API_KEY=<your-api-key>
```

Then run:

```
node --env-file=.env refill.ts
```

Open your doc — there's a new row in **Incoming Orders** with line items for each low-stock product.